DNSHawk: PySide6 migration and Windows `.exe` deployment plan

We'll replace the Tkinter interface with a PySide6 + Qt Widgets desktop application, using a dark security-console design, while preserving DNSHawk's existing detection backend.

The goal is a Windows application that a user can launch, configure, and operate without manually running Python commands or inspecting every DNS event.

Core principle: rebuild the presentation layer, not the detection engine. The existing 60-second D1 feature pipeline, XGBoost model, rule detector, TShark capture, DNS parser, and Kali testing workflow remain the foundation.

1. Target application design

DNSHawk

DNS Security Monitoring

READY

DNS queries

Live counter

Open incidents

Alert queue

Navigation

Overview

Incidents

Traffic Explorer

Investigation

Capture & Settings

Conceptual layout, not a live application screenshot. Actual counts and statuses will come from the running application.

The Overview screen will show the capture state, event volume, incidents requiring attention, and recent detections. The Incidents view will summarize suspicious behavior

automatically. Raw events and feature values will remain available in Traffic Explorer and Investigation.

2. Preserve the existing backend

The new GUI should call the existing project modules directly rather than reimplementing their logic.

PySide6 presentation layer

Overview · Incidents · Traffic · Investigation · Settings

Capture worker + event queue

Capture runs outside the GUI thread; UI updates occur on the Qt main thread.

Existing DNSHawk backend

TSharkCapture → DNS parser → DNSEvent → WindowManager / DNSWindow → FeatureEngine → RuleDetector + MLDetector

Session and incident services

Incident grouping · Export · Persistent logs · Session history

We'll preserve these existing modules:

- `dnshawk/collector/tshark_capture.py`
- `dnshawk/parser/dns_parser.py`
- `dnshawk/models/dns_event.py`
- `dnshawk/models/dns_window.py`
- `dnshawk/aggregation/window.py`
- `dnshawk/features/feature_engine.py`
- `dnshawk/detection/rule_detector.py`
- `dnshawk/detection/ml_detector.py`

Kali Test mode will also be retained. The five-minute behavioral detector is a later enhancement, not a dependency for the initial GUI migration.

3. Proposed project structure

I recommend introducing a new GUI package while leaving the existing backend files untouched.

```
DNSHawk/
├── main.py
├── dnshawk/
```

```
      │   ├── collector/
      │   ├── parser/
      │   ├── models/
      │   ├── aggregation/
      │   ├── features/
      │   ├── detection/
      │   ├── gui/                  # Existing Tkinter GUI
      │   └── qt_gui/               # New PySide6 GUI
      │       ├── app.py
      │       ├── main_window.py
      │       ├── workers/
      │       │   └── capture_worker.py
      │       ├── views/
      │       │   ├── overview.py
      │       │   ├── incidents.py
      │       │   ├── traffic_explorer.py
      │       │   ├── investigation.py
      │       │   └── settings.py
      │       ├── services/
      │       │   ├── incident_service.py
      │       │   └── session_service.py
      │       └── widgets/
      ├── model_training/
      │   └── DNSHawk_D1_artifacts/
      │       └── models/
      │           └── xgboost_dns_tunnel_d1.json
      ├── assets/
      │   └── dnshawk.ico
      ├── tests/
      ├── requirements.txt
      └── packaging/
```

This is the target structure, not a claim that these new files have already been created.

4. Windows executable and external dependencies

For packaging, I recommend starting with PyInstaller in one-folder mode, then evaluating a single-file build. Qt for Python also provides `pyside6-deploy`, which uses Nuitka and supports standalone and one-file deployment.



Qt for Python

+2

Component	Deployment decision
PySide6 / Qt	Bundle the required Qt libraries and plugins
Python dependencies	Bundle compatible versions of XGBoost, NumPy and other imported packages
D1 model	Bundle the JSON model as a read-only resource
TShark	Detect a supported installatio

	n; initially document or provide an installer prerequisite
Capture driver	Check for the required capture driver and permissions
Reports and logs	Write to a persistent user-writable directory
Configuration	Store separately from the executable so it survives upgrades

TShark deserves special treatment

The GUI can be bundled into an `.exe`, but TShark is an external executable with its own runtime dependencies. We should not assume that including the Python code will make packet capture work on another computer.

On startup, DNSHawk should:

1. Locate `tshark.exe`, using a configured path or a supported installation path.
2. Run a version check and verify that interfaces can be enumerated.
3. Display actionable errors if TShark or the capture driver is missing.
4. Explain when administrator privileges or other Windows permissions are needed.
5. Provide a capture diagnostic screen before the user starts monitoring.

Initially, I'd use an installed Wireshark/TShark prerequisite rather than trying to redistribute the entire capture stack. We can investigate a bundled deployment later, including its licensing and driver-installation implications.

Model and persistent paths

The D1 model should be loaded from a resource path that works both in development and after packaging. Logs, CSV exports and configuration must never be written into the temporary extraction directory of a one-file executable. PyInstaller documents that one-file applications unpack bundled resources into a temporary directory, so application data needs a separate persistent location.



PyInstaller 6.22.3 documentation

+1

For example:

```
%LOCALAPPDATA%\DNSHawk\
├── config\
│   └── settings.json
├── logs\
├── sessions\
├── exports\
└── diagnostics\
```

The D1 model remains bundled and read-only by default; the user's logs, settings and exported reports remain outside the bundle.

5. Implementation sequence and acceptance tests

1. Create the Qt application shell. Set up PySide6, the dark theme, navigation, status bar and application logging.
   Acceptance: the app launches without Tkinter and resizes correctly.
2. Port live capture and Kali Test mode. Use a worker thread and Qt signals for events, errors and status changes.
   Acceptance: start/stop works repeatedly; the GUI remains responsive; capture errors reach the interface.
3. Integrate the existing detectors. Reuse `FeatureEngine`, `RuleDetector` and `MLDetector`, retaining the 60-second windows and the D1 minimum of 10 queries for ML classification.
   Acceptance: regression tests confirm that identical input windows produce equivalent backend results.
4. Build the automatic incident workflow. Add severity prioritization, related-event grouping, explanations and investigation views.
   Acceptance: users can understand a detection without manually opening each event.
5. Add session management. Include Clear Session, persistent logs, CSV/JSON exports, and safe shutdown.
   Acceptance: clearing the display doesn't silently delete saved evidence.
6. Package and test on Windows. Build the executable, test it on the development machine, then test on a clean Windows environment without a development Python environment.
   Acceptance: model loading, interface discovery, packet capture, export paths, and graceful failure all work in the packaged application.

6. What we need before implementation

I have the project structure and earlier GUI context, but I don't have a verified copy of your current complete project tree and backend files in this turn. Before writing the migration, we should inspect the current files so the new GUI uses the actual APIs rather than assumptions.

The first implementation milestone will be a runnable PySide6 shell integrated with your existing capture and detection pipeline. We will keep the Tkinter GUI intact until the Qt version passes the regression tests, then make PySide6 the default entry point.

That gives us a controlled migration to a distributable `DNSHawk.exe` without risking the detection work you've already completed.